

Arquitetura & Client-Side

Arquitetura Web & Programação Client-Side

Prof.^a Catarina Silva

Ano Letivo 2025/2026 – 2º Semestre

Objetivos da Aula

- Estruturar o código JavaScript utilizando **Funções**.
- Compreender e manipular a árvore **DOM** (Document Object Model).
- Tornar a página interativa através da escuta de **Eventos**.
- Distinguir execução Síncrona de **Assíncrona**.
- Compreender o formato de dados **JSON**.
- Realizar pedidos de rede sem recarregar a página utilizando **AJAX** e a **Fetch API**.

Agenda

1. Funções em JavaScript
2. A Árvore DOM
3. Eventos e Interatividade
4. Comunicação Assíncrona e JSON
5. AJAX e a Fetch API

O que é uma Função?

- É um bloco de código desenhado para executar uma tarefa específica.
- **Vantagens:**
 - Reutilização de código (*Princípio DRY - Don't Repeat Yourself*).
 - Divisão de problemas complexos em partes menores e geríveis.
 - Organização e facilidade de manutenção.
- Uma função só é executada quando é invocada (chamada).

Sintaxe Básica

- As funções podem receber dados (parâmetros/argumentos) e podem devolver um resultado (através do return).

```
// 1. Declaracao da funcao
function calcularArea(largura, altura) {
    let area = largura * altura;
    return area;
}

// 2. Invocacao da funcao
let resultado = calcularArea(5, 10);
console.log("A area e: " + resultado); // Output: 50
```

Revisão: O que é o DOM?

- Document **O**bject **M**odel.
- É a representação em memória que o browser faz do documento HTML.
- O HTML é transformado numa estrutura em **árvore**, onde cada tag é um "Nó"(Objeto).
- O JavaScript usa o objeto global document para navegar, ler e alterar esta árvore em tempo real.

Passo 1: Selecionar Elementos

- Para modificar algo, o JS precisa primeiro de o encontrar no HTML.

```
// Selecionar pelo ID unico
let titulo = document.getElementById("meuTitulo");

// Selecionar o primeiro elemento que coincida com o seletor CSS
let botao = document.querySelector(".btn-primario");

// Selecionar todos os elementos (devolve uma NodeList)
let paragrafos = document.querySelectorAll("p");
```

Passo 2: Manipular Conteúdo e Estilos

- Uma vez selecionado, podemos alterar as propriedades do elemento.

```
let caixa = document.querySelector(".caixa-aviso");

// Alterar o texto interno
caixa.textContent = "Aviso importante!";

// Alterar o HTML interno
caixa.innerHTML = "<strong>Aviso:</strong> Sistema em baixo.";

// Alterar o estilo CSS diretamente
caixa.style.backgroundColor = "red";
caixa.style.color = "white";
```


Passo 3: Criar e Inserir Elementos

- O JS também consegue construir HTML de raiz e inseri-lo na página.

```
// 1. Criar um novo elemento (ainda so existe na memoria)
let novoItem = document.createElement("li");
novoItem.textContent = "Sumo de Laranja";

// 2. Encontrar o local onde o queremos colocar
let listaCompras = document.getElementById("lista");

// 3. Adicionar o novo elemento como "filho" da lista
listaCompras.appendChild(novoItem);
```

O que são Eventos?

- Eventos são "coisas" que acontecem aos elementos HTML.
- Exemplos: O utilizador clica num botão, passa o rato numa imagem, escreve num *input*, ou o browser termina de carregar a página.
- O JavaScript permite-nos "escutar" estes eventos e executar código (uma função) em resposta.

Escutar Eventos (addEventListener)

- É a forma mais moderna e recomendada de associar ações a eventos.

```
let meuBotao = document.getElementById("btnGuardar");

// addEventListener(tipoDeEvento, funcaoAExecutar)
meuBotao.addEventListener("click", function() {
    alert("O botao foi clicado!");
    // Podiamos chamar uma funcao pre-definida aqui
});
```

O Objeto Event (e)

- Quando um evento ocorre, o JS passa automaticamente um objeto com informações sobre esse evento para a nossa função.
- Muito útil em formulários!

```
let form = document.getElementById("meuFormulario");

form.addEventListener("submit", function(e) {
    // Impede o comportamento padrao (recarregar a pagina)
    e.preventDefault();

    console.log("Formulario submetido em JS!");
});
```

Síncrono vs. Assíncrono

- **Código Síncrono:** Executa linha a linha. Se uma linha demorar muito tempo (ex: transferir uma imagem pesada), o browser "congela" até terminar.
- **Código Assíncrono:** Inicia uma tarefa demorada em "segundo plano" e passa imediatamente para a linha seguinte. Quando a tarefa terminar, o JS é avisado e processa o resultado.
- **Porquê?** Na Web, pedir dados a servidores externos demora tempo. O assincronismo garante que a página continua interativa.

O que é o JSON?

- JavaScript **O**bject **N**otation.
- O formato *standard* da Web moderna para transferir dados entre um Cliente (Browser) e um Servidor (API).
- É simples texto, leve e independente da linguagem de programação.
- É composto por pares **Chave-Valor** (semelhante a um dicionário).
- **Regra de Ouro:** As chaves e textos têm de usar aspas duplas .

Anatomia de um JSON

```
{  
  "nome": "João Silva",  
  "idade": 21,  
  "estudante": true,  
  "disciplinas": ["Matematica", "Programacao"],  
  "morada": {  
    "cidade": "Aveiro",  
    "pais": "Portugal"  
  }  
}
```

Trabalhar com JSON em JavaScript

- O JS possui o objeto global JSON para converter de/para texto.

```
// Receber dados do servidor (String JSON -> Objeto JS)
let textoJson = '{"aluno": "Ana", "nota": 18}';
let dadosObj = JSON.parse(textoJson);
console.log(dadosObj.aluno); // Ana

// Enviar dados para o servidor (Objeto JS -> String JSON)
let meuObjeto = { produto: "Rato", preco: 25 };
let textoParaEnviar = JSON.stringify(meuObjeto);
```


O que é AJAX?

- **Asynchronous JavaScript and XML.**
- É uma técnica (não uma linguagem) que permite ao browser pedir e enviar dados ao servidor **depois** da página já estar carregada.
- **A Grande Vantagem:** Atualiza partes específicas do ecrã sem ter de dar "Refresh" à página toda (Ex: carregar mais posts numa rede social, pesquisa do Google).
- Hoje em dia, usa-se JSON em vez de XML na vasta maioria dos casos.

A Fetch API

- É a interface moderna do JavaScript para realizar pedidos AJAX.
- Baseia-se no conceito de **Promises** (Promessas).
- Uma Promise é um objeto que representa um pedido em curso: pode vir a ser "resolvida"(sucesso) ou "rejeitada"(erro na ligação).
- Lidamos com o sucesso usando o método `.then()` e com os erros usando o `.catch()`.

O Ciclo do Fetch (Pedido GET)

```
// 1. Fazer o pedido assincrono ao Servidor
fetch('https://api.exemplo.com/usuarios')

    // 2. Quando o servidor responder, converter para JS
    .then(function(resposta) {
        return resposta.json();
    })

    // 3. Quando a conversao terminar, usar os dados
    .then(function(dados) {
        console.log(dados);
        // Manipular o DOM aqui!
    })

    // 4. Se a internet falhar, capturamos o erro
    .catch(function(erro) {
        console.error("Erro na ligacao:", erro);
    });
```